Naresh Edagotti

Follow For More

50

Transformer Interview

Questions and Answers

PracticAI

1.What problem were Transformers originally designed to solve in seq2seq models, and how do they eliminate the need for recurrence or convolution?

Answer: Transformers were designed to solve the sequential processing bottleneck in RNNs. Traditional RNNs process tokens one by one, which is slow and struggles with long-range dependencies. Transformers use attention mechanisms that process all tokens in parallel, eliminating the need for recurrence.

For example, when translating "The cat that chased the mouse ran away", an RNN must process each word sequentially and pass information through 8 sequential steps. A Transformer directly connects "cat" and "ran" through attention, computing all relationships simultaneously. This parallel processing makes training 10x faster and better captures dependencies between distant words. No convolution or recurrence is needed because attention directly models relationships between any two positions.

2.Walk through the full data flow inside a vanilla Transformer from input tokens to output predictions.

Answer: The flow starts with tokenizing input text into tokens. These tokens become embeddings (dense vectors) and get added with positional encodings. The combined embeddings pass through N encoder layers. Each encoder layer has self-attention (where each token attends to all tokens) followed by a feedforward network, with residual connections and layer normalization.

For example, encoding "Hello world" produces contextual representations. The decoder takes target tokens (like "Hola"), adds positional encodings, and processes through N decoder layers. Each decoder layer has masked self-attention (only sees previous tokens), cross-attention (attends to encoder outputs), and feedforward network. Finally, a linear projection and softmax produce probability distribution over vocabulary to predict the next token. This repeats autoregressively to generate the full output sequence.

3.Why do Transformers rely on embeddings and what is the difference between token embeddings, positional embeddings and segment embeddings?

Answer: Transformers need continuous vector representations to perform mathematical operations, so discrete tokens are converted to embeddings. Token embeddings capture semantic meaning where similar words have similar vectors. Positional embeddings add sequence order information since attention itself is order-agnostic. Segment embeddings distinguish different parts of input.

For example, the word "bank" gets a token embedding representing its meaning. Positional embedding adds information that "bank" is at position 3 in the sentence. In BERT, if you have two sentences "I like cats. Dogs are nice.", segment embedding marks the first sentence as segment A and second as segment B. All three are added together: final representation = token embedding + positional embedding + segment embedding. This combined vector feeds into the Transformer layers.

4.Explain positional encoding. Why can't Transformers understand order without it? Compare sinusoidal vs learned positional embeddings.

Answer: Attention mechanisms treat input as an unordered set because they compute relationships between all pairs simultaneously. Without positional encoding, "cat chased mouse" and "mouse chased cat" would look identical to the model. Positional encodings inject order information by adding position-specific vectors to token embeddings.

For example, sinusoidal encodings use fixed sine and cosine functions at different frequencies: PE(pos, 2i) = sin(pos/10000^(2i/d)). They generalize to longer sequences than seen during training. Learned positional embeddings are trained parameters, one for each position. Learned embeddings can adapt better to specific tasks but cannot

extrapolate to longer sequences. Modern models like GPT use learned embeddings for fixed context windows, while models needing length generalization use sinusoidal or rotary positional encodings (RoPE).

5.What are residual connections and why are they placed around attention and feedforward blocks?

Answer: Residual connections add the input of a layer directly to its output: output = Layer(x) + x. They solve the vanishing gradient problem in deep networks by creating direct paths for gradients to flow backward. Without residuals, gradients can become extremely small when backpropagating through many layers, making training impossible.

For example, in a 24-layer Transformer, gradients must flow through 24 attention and feedforward blocks. With residual connections, gradients can skip layers and flow directly backward, maintaining strong gradient signals. This allows training very deep models. The residual path also helps preserve original information. If a layer learns something harmful, the residual connection ensures the original input still passes through, acting as a safety mechanism. Modern Transformers place residuals around both attention and feedforward sublayers.

6.What happens inside LayerNorm? Why is it applied before sublayers in modern architectures (Pre-LN) instead of after (Post-LN)?

Answer: LayerNorm normalizes activations across features for each token independently. It computes mean and variance across the embedding dimension, then normalizes: output = (x - mean) / sqrt(variance + epsilon) * gamma + beta. This stabilizes training by keeping activation scales consistent. Gamma and beta are learnable parameters.

For example, if token embeddings have values [100, 200, 150], LayerNorm rescales them to have mean 0 and variance 1. Pre-LN (applying normalization before attention/FFN) provides more stable training for deep models because gradients flow through fewer transformations. Post-LN (normalization after) was used in original Transformer but causes training instability in very deep networks. Pre-LN acts as a preconditioner, ensuring each layer receives well-scaled inputs. Most modern models like GPT-3 use Pre-LN because it enables training deeper architectures without careful tuning.

7.Describe the role of the Feed Forward Network (FFN). Why is it applied independently to each token?

Answer: The FFN is a two-layer neural network applied to each token position independently. It consists of two linear transformations with a nonlinear activation (usually GELU or ReLU) in between: FFN(x) = W2 * GELU(W1 * x + b1) + b2. The first layer typically expands dimension 4x, then the second projects back to original size.

For example, if embedding size is 768, FFN expands to 3072 (intermediate size), then back to 768. Each token is processed independently because FFN adds nonlinear transformations to enrich representations after attention mixes information between tokens. Attention gathers context from other tokens, while FFN processes each token's enriched representation individually. This position-wise application allows efficient parallel computation. The expansion and compression pattern helps the model learn complex feature transformations while keeping parameter count manageable.

8.Walk through the math of scaled dot-product self-attention. Why do we scale by square root of d_k?

Answer: Self-attention computes three matrices: Query (Q), Key (K), and Value (V) by multiplying input X with learned weight matrices. Attention scores are computed as: Attention(Q, K, V) = softmax(Q * K^T / sqrt(d_k)) * V. The dot product Q * K^T measures similarity between tokens, softmax converts to probabilities, then these weights are applied to values.

For example, with tokens "cat sat mat" and d_k=64, Q * K^T produces a 3x3 similarity matrix. Without scaling, dot products can become very large (imagine two 64-dimensional vectors, their dot product can be huge). Large values

push softmax into regions with tiny gradients, causing training problems. Dividing by sqrt(64)=8 keeps values in a reasonable range. The square root specifically maintains variance: if Q and K have unit variance, Q * K^T has variance d_k, so dividing by sqrt(d_k) normalizes it back.

9.How does multi-head attention help the model learn richer relationships compared to single-head attention?

Answer: Multi-head attention runs multiple attention operations in parallel, each with different learned projections. Instead of one set of Q, K, V matrices, you have h heads, each learning different relationship patterns. Outputs are concatenated and linearly transformed: MultiHead(Q,K,V) = Concat(head1, ..., headh) * W^O where each head_i = Attention(Q*W_i^Q, K*W_i^K, V*W_i^V).

For example, with 8 heads and embedding size 512, each head operates in 64 dimensions (512/8). One head might learn syntactic relationships (subject-verb), another semantic similarity (synonyms), another positional proximity (neighboring words). In "The quick brown fox jumped", one head connects "fox" to "jumped" (subject-verb), another connects "quick" and "brown" to "fox" (modifiers), giving richer understanding than single attention. This parallel diversity lets the model capture multiple types of relationships simultaneously without interference.

10.Compare self-attention, cross-attention and masked self-attention. Where is each used?

Answer: Self-attention has tokens attend to other tokens in the same sequence. All Q, K, V come from the same input. Cross-attention has queries from one sequence and keys/values from another sequence. Masked self-attention prevents tokens from attending to future positions by setting future attention scores to negative infinity before softmax.

For example, in encoder (BERT-style), self-attention lets "cat" attend to all words in "the cat sat". In decoder during generation, masked self-attention ensures when predicting word 3, you only see words 1 and 2, not future words. Cross-attention in encoder-decoder models (like translation) lets decoder attend to encoder outputs, so when generating "Hola", the decoder queries can attend to all encoded representations of "Hello". Self-attention is used in encoders, masked self-attention in decoder self-attention layers, and cross-attention in decoder cross-attention layers.

11.Why do attention matrices grow quadratically with sequence length? What real problems does this create in production?

Answer: Attention computes relationships between every pair of tokens. For sequence length n, this creates an n x n attention matrix. Memory and computation both scale as O(n^2). With 1000 tokens, you need a 1,000,000-element matrix. With 10,000 tokens, it becomes 100,000,000 elements.

For example, processing a 4096-token document requires storing a 16,777,216-element attention matrix per head. With 32 heads, that is over 500 million values just for one layer. This causes: 1) GPU memory exhaustion beyond certain lengths (typical limit 2048-8192 tokens), 2) slow inference (computing 16M similarities takes time), 3) high costs (longer sequences cost much more). Solutions include sparse attention (only attend to subset of tokens), sliding window attention (local attention only), or linear attention approximations. Models like Longformer use sliding windows plus global attention for long documents.

12.What are key, query and value vectors? Why do we transform embeddings into K, Q and V instead of directly attending over raw embeddings?

Answer: In attention, queries are what you are looking for, keys are what each token offers, and values are what you retrieve. These are created by multiplying token embeddings with three learned weight matrices W^Q, W^K, W^V. We transform embeddings because this allows the model to learn task-specific representations for matching and retrieval.

For example, for the word "bank" in "river bank", the query might encode "looking for geographical context", the key encodes "I am a geographical location", and the value contains the actual geographical features. Using raw embeddings would force the same representation to serve all three roles. Separate transformations let the model learn: Q to represent information needs, K to represent what information is available, V to represent the actual information content. This flexibility is crucial because the same word needs different representations for matching versus content retrieval.

13.Describe the encoder architecture. What does each encoder block learn?

Answer: The encoder consists of N identical layers (typically 6-12), each containing two sublayers: multi-head self-attention and position-wise feedforward network. Each sublayer has residual connections and layer normalization. The encoder processes the entire input sequence in parallel, with each layer refining token representations.

For example, processing "The cat sat on the mat", layer 1 might learn basic word relationships and syntax. Layer 2 builds on this to understand "cat" is the subject and "mat" is the location. Deeper layers capture more abstract relationships like the complete scene description. Each encoder block transforms the 512-dimensional vectors (or whatever embedding size) while maintaining the same dimensionality. The final encoder output is a sequence of context-aware representations where each token's vector contains information from the entire input sequence, used by the decoder for generation tasks or directly for classification tasks.

14.Describe the decoder architecture. Why does decoder require both masked self-attention and encoder-decoder cross attention?

Answer: The decoder has N identical layers, each with three sublayers: masked self-attention, encoder-decoder cross-attention, and feedforward network. Masked self-attention is needed because the decoder generates tokens autoregressively (one at a time) and cannot see future tokens during training or inference. Cross-attention is needed to incorporate source information from the encoder.

For example, when translating "Hello world" to "Hola mundo", at the step generating "mundo", the decoder uses: 1) Masked self-attention to look at previously generated tokens ("Hola"), learning from its own partial output. 2) Cross-attention to attend to encoder representations of "Hello world", understanding what source content to translate next. Without masking, the model would cheat during training by seeing "mundo" when predicting "mundo". Without cross-attention, the decoder would have no way to access source sentence information and could not perform translation.

15.How does the decoder generate tokens autoregressively? Explain teacher forcing and exposure bias.

Answer: Autoregressive generation means producing one token at a time, where each new token depends on all previously generated tokens. The decoder takes the sequence generated so far, processes it, and outputs probability distribution for the next token. Teacher forcing means during training, the decoder receives ground-truth previous tokens as input, not its own predictions.

For example, when training to output "Hola mundo", at step 2, teacher forcing feeds the decoder the correct "Hola" to predict "mundo", even if the model would have wrongly predicted "Hello". This makes training faster and more stable. However, it creates exposure bias: during training the model always sees correct context, but during inference it sees its own predictions, which might be wrong. If the model predicts "Buenos" instead of "Hola", it has never trained on generating the second word after "Buenos", leading to error accumulation. Solutions include scheduled sampling (sometimes using model predictions during training) or using inference-time decoding strategies.

16.In what cases would you use only the encoder (e.g., BERT), only the decoder (GPT), or both (T5, original Transformer)?

Answer: Use encoder-only for tasks requiring understanding and classification, where you need bidirectional context. Use decoder-only for generative tasks that proceed left-to-right. Use encoder-decoder for sequence-to-sequence tasks that transform one sequence into another.

For example, BERT (encoder-only) is ideal for sentiment analysis, named entity recognition, or question answering where understanding the full context helps. You input "This movie was amazing" and classify sentiment. GPT (decoder-only) excels at text generation, completion, and few-shot learning because it naturally models the probability of next tokens. You input "Once upon a time" and generate a story. T5 and original Transformer (encoder-decoder) work best for translation, summarization, or question answering where you consume one sequence and produce another. You input an English sentence and output Spanish translation. Encoder-decoder is more parameter-efficient for such tasks than decoder-only models.

17.Compare BPE, WordPiece, and SentencePiece. Why are subword tokenizers crucial for multilingual systems?

Answer: BPE (Byte Pair Encoding) starts with characters and iteratively merges the most frequent pairs. WordPiece is similar but uses likelihood maximization for merging. SentencePiece treats input as raw text without pre-tokenization and uses BPE or unigram language model. All create subword vocabularies that handle rare words and morphology better than word-level tokenization.

For example, BPE might split "unhappiness" into "un", "happi", "ness" if these are common subwords. This handles "unhappily" or "happiness" without separate vocabulary entries. For multilingual systems, subword tokenizers are crucial because they share subwords across languages. The prefix "un-" in English and "in-" in Spanish both indicate negation and can share representations. Languages with rich morphology like German ("Donaudampfschifffahrtsgesellschaft") can be broken into meaningful components. This reduces vocabulary size from millions to 30,000-50,000 tokens while handling any word through subword composition, making multilingual models feasible.

18.What is embedding projection weight tying? Why does it improve performance and reduce parameters?

Answer: Weight tying means using the same weight matrix for both input token embeddings and the final output projection layer. Instead of two separate matrices (one to embed input tokens, one to project decoder output to vocabulary), you use one shared matrix and its transpose. This typically reduces parameters by 30-50% in the vocabulary-related layers.

For example, in a model with 50,000 vocabulary and 768 dimensions, the embedding matrix has 38.4 million parameters. Without tying, the output projection adds another 38.4 million, totaling 76.8 million. With weight tying, you only have 38.4 million. This works because both operations involve token-vector relationships: embeddings map tokens to vectors, output projection maps vectors back to token probabilities. The shared constraint acts as regularization, forcing the model to learn a consistent representation space. Empirically, it improves perplexity and reduces overfitting, especially for smaller models or limited training data.

19.Explain Bi-Encoder architecture. When is it preferred over Cross-Encoder? Give examples from RAG systems.

Answer: Bi-Encoder uses two separate encoders to independently encode two inputs into fixed-size embeddings, then compares them using similarity metrics like cosine similarity. It processes each input once and can precompute embeddings. Cross-Encoder concatenates both inputs and encodes them jointly with attention between them, producing a single relevance score.

For example, in RAG (Retrieval Augmented Generation), Bi-Encoder is used for first-stage retrieval. You encode millions of documents offline into embeddings stored in a vector database. At query time, you encode the query and

find nearest neighbors using fast approximate search (like FAISS). This is efficient because document encodings are reused. You might retrieve 100 candidates in milliseconds. Then a Cross-Encoder re-ranks the top candidates by encoding query-document pairs jointly, which is slower but more accurate. Bi-Encoder enables scalable search over millions of documents, while Cross-Encoder provides precise ranking for a small candidate set.

20.Explain Cross-Encoder architecture. Why is it more accurate but slower? Give examples like re-rankers.

Answer: Cross-Encoder takes two texts as input, concatenates them with a separator token, and encodes them jointly with full attention between both inputs. The model outputs a single relevance or similarity score. This allows modeling complex interactions between the inputs but requires encoding every pair individually.

For example, in a search re-ranker, you have query "best Italian restaurants" and a document "Mario's Pizzeria offers authentic Italian cuisine in downtown". Cross-Encoder concatenates them as "[CLS] best Italian restaurants [SEP] Mario's Pizzeria offers..." and processes with full attention. The word "Italian" in the query can directly attend to "Italian" in the document, capturing exact matching. "Best" can attend to "authentic", capturing semantic alignment. This deep interaction makes it more accurate than Bi-Encoder similarity scores. However, with 100 candidate documents, you need 100 separate encoding operations. This is too slow for initial retrieval over millions of documents, so Cross-Encoders are used as re-rankers for small candidate sets.

21.How does a multilingual encoder model (XLM-R, mBERT) manage to represent different languages in one embedding space?

Answer: Multilingual encoders are trained on text from many languages simultaneously using masked language modeling. The shared subword vocabulary includes pieces from all languages, and the model learns to map similar concepts across languages to nearby points in the embedding space. This happens naturally through shared vocabulary, cognates, and parallel structure.

For example, XLM-R trained on 100 languages learns that "cat", "gato" (Spanish), "chat" (French), and "Katze" (German) should have similar representations because they appear in similar contexts: "The ___ sat on the mat". Subword overlap helps: "information" and "información" share "informa". Numbers, proper nouns, and punctuation are universal anchors. The model learns a shared semantic space where translation equivalents cluster together. At test time, you can encode French text and use it for a task trained on English examples because equivalent concepts have similar representations. This zero-shot cross-lingual transfer makes multilingual models powerful for low-resource languages.

22.How is a multilingual encoder-decoder model trained to translate many language pairs without pairwise datasets?

Answer: Multilingual encoder-decoder models like mBART or mT5 are trained with denoising objectives on monolingual data in many languages, then fine-tuned on translation data. They use special language tags to indicate source and target languages. The model learns a shared multilingual representation space where the encoder understands any input language and the decoder can generate any target language.

For example, mBART is pre-trained by corrupting sentences in 25 languages and learning to reconstruct them. This teaches both encoder and decoder to work with all languages. Then it is fine-tuned on translation pairs like English-German and French-Spanish. When you add a tag like "translate to Spanish:", it learns to route through the shared space. For translating Tamil to Korean (even without direct training pairs), the model can translate via the shared representation: encode Tamil into language-neutral space, decode from that space to Korean. This many-to-many translation works through the pivot representation learned during multilingual pre-training and fine-tuning on available language pairs.

23.What is mixture-of-experts (MoE)? How do gating networks route tokens? What are the challenges like expert imbalance?

Answer: MoE replaces dense feedforward layers with multiple expert networks (FFNs), where only a subset of experts process each token. A gating network decides which experts to activate per token. This allows scaling model capacity without proportionally increasing computation. The gating network computes routing probabilities and typically selects top-k experts (like k=2).

For example, in a layer with 8 experts, the gating network might route "Tokyo" to experts 2 and 5 (which handle geographical entities), while "happiness" goes to experts 3 and 7 (which handle abstract concepts). Only 2 out of 8 experts compute for each token, reducing cost by 4x compared to using all experts. Challenges include expert imbalance where some experts get overloaded while others are underused, creating bottlenecks. Solutions include load-balancing losses that encourage even distribution, auxiliary losses that penalize routing entropy, and capacity factors that limit tokens per expert. MoE models can be extremely large (like Switch Transformer with 1.6 trillion parameters) while keeping per-token cost manageable.

24.Compare dense Transformers vs MoE Transformers in terms of speed, scaling and cost.

Answer: Dense Transformers activate all parameters for every token, giving consistent performance but linear cost scaling with model size. MoE Transformers activate only a subset of parameters per token through expert routing, enabling much larger total capacity with sublinear cost scaling. Training speed, inference cost, and model size trade-offs differ significantly.

For example, a dense Transformer with 10 billion parameters uses all 10B parameters for every token, costing 10B FLOPs per token. An MoE model with 100 billion total parameters but 2-of-8 expert routing might use only 15B parameters per token (shared layers plus 2 experts), costing 15B FLOPs while having 10x total capacity. MoE models scale to trillions of parameters while maintaining reasonable speed. However, MoE has challenges: 1) communication overhead when distributing experts across GPUs, 2) memory requirements to store all experts even if not all are used, 3) load balancing complexity, 4) harder to deploy in production. Dense models are simpler and more predictable, while MoE enables massive scale for training but requires careful engineering.

25.What is label smoothing and why is it useful in sequence-to-sequence training?

Answer: Label smoothing replaces hard one-hot target labels with soft labels that assign small probability to incorrect classes. Instead of target distribution [0, 0, 1, 0, 0] for the correct token, you might use [0.02, 0.02, 0.92, 0.02, 0.02]. This prevents overconfidence and acts as regularization.

For example, when training translation, the model might output "good" or "great" both being reasonable translations of "bueno". With hard labels, the model is forced to put 100% probability on "good" even though "great" is also valid. This causes overfitting and poor calibration. Label smoothing with epsilon=0.1 assigns 90% to correct token and distributes 10% among others, encouraging the model to not be overly confident. This improves generalization, reduces overfitting to noisy labels, and produces better calibrated probability estimates. It slightly hurts training loss but consistently improves validation performance and translation quality metrics like BLEU.

26.Why do Transformers often use Adam or AdamW with warmup and cosine decay scheduling?

Answer: Adam and AdamW use adaptive learning rates per parameter based on first and second moment estimates of gradients. AdamW adds decoupled weight decay for better regularization. Transformers benefit from warmup because large learning rates at the start with random initialization can cause training instability. Cosine decay gradually reduces learning rate allowing fine-tuning.

For example, you might start with learning rate 0 and linearly increase to 0.001 over 4000 steps (warmup). This prevents early gradient explosions when parameters are random. Then cosine decay reduces learning rate following a cosine curve from 0.001 to 0.0001 over remaining training. Without warmup, large initial updates can push parameters into bad regions with vanishing gradients. Without decay, the model might not fine-tune well in later training. AdamW's decoupled weight decay prevents overfitting better than Adam's L2 regularization. This combination has become standard because it reliably trains models ranging from small BERT to massive GPT models without extensive tuning.

27.How does gradient checkpointing help memory efficiency during training?

Answer: Gradient checkpointing saves memory by not storing all intermediate activations during the forward pass. Instead, it only saves checkpoints at certain layers. During backpropagation, it recomputes the missing activations on the fly. This trades computation time for memory space.

For example, in a 24-layer transformer, you might save activations only at layers 6, 12, 18, and 24. When computing gradients, you recompute layers 1-5 from checkpoint at layer 6. This lets you train larger models or use bigger batch sizes without running out of GPU memory. The cost is about 20-30% slower training, but you can fit models that wouldn't work otherwise.

28.What are rotary embeddings (RoPE) and why do many modern models use them instead of fixed sinusoidal embeddings?

Answer: RoPE (Rotary Position Embeddings) encode position information by rotating the query and key vectors in attention. Unlike fixed sinusoidal embeddings that add position info, RoPE multiplies position into the attention mechanism itself.

The key advantage is better extrapolation to longer sequences. If trained on 2048 tokens, RoPE can handle 4096 or 8192 tokens reasonably well. Fixed embeddings struggle beyond training length. For example, models like LLaMA and GPT-NeoX use RoPE because it maintains relative position relationships better. When token A is 5 positions before token B, RoPE preserves this relationship regardless of their absolute positions in the sequence.

29.What is contrastive learning and how is it used to train bi-encoders?

Answer: Contrastive learning trains models by pulling similar examples together and pushing dissimilar ones apart in embedding space. For bi-encoders, you encode queries and documents separately, then train them to have similar embeddings for positive pairs and different embeddings for negative pairs.

For example, in sentence similarity, given query "best Italian restaurant" with positive document "top Italian dining spots" and negative document "Chinese takeout menu", you train so that query-positive distance is small and query-negative distance is large. The loss function (like InfoNCE) maximizes similarity scores for correct pairs while minimizing scores for incorrect pairs. This creates an embedding space where semantically similar items cluster together.

30.Compare greedy decoding, beam search, top-k sampling and nucleus sampling. When do you use each?

Answer: Greedy decoding picks the highest probability token at each step. Fast but repetitive. Use for deterministic tasks like translation.

Beam search keeps top B candidates at each step, exploring multiple paths. Better quality than greedy. Use for tasks needing accuracy like summarization.

Top-k sampling randomly samples from top k tokens. More diverse than greedy. Use for creative writing where variety matters.

Nucleus (top-p) sampling samples from smallest set of tokens whose cumulative probability exceeds p (like 0.9). Adapts to confidence. Use for open-ended generation like chatbots.

Example: For "The cat sat on the...", greedy always picks "mat", beam search considers "mat/floor/chair", top-k might pick from top 50 words, nucleus adapts based on distribution.

31.Why do long context models struggle with attention over very long sequences? Explain recent solutions like sliding-window attention, sparse attention, and linear attention methods.

Answer: Standard attention has $O(n^2)$ complexity in sequence length. For 10,000 tokens, that's 100 million comparisons. This is slow and memory intensive.

Sliding-window attention only attends to nearby tokens (like 512 tokens before and after). Fast but loses long-range dependencies. Used in Longformer.

Sparse attention attends to selected positions using patterns (every k-th token, random positions). Reduces computation while keeping some long-range connections. Used in BigBird.

Linear attention approximates attention using kernel methods, achieving O(n) complexity. Faster but sometimes less accurate. Used in models like Performer.

Example: For 8k token document, sliding window processes locally, sparse attention connects key anchor points, linear attention approximates full attention efficiently.

32.If your translation model keeps producing repeated words, which part of the Transformer would you examine first and why?

Answer: Check the **decoder's self-attention mechanism** first. Repetition often happens when attention weights collapse to focus too much on recently generated tokens, creating a feedback loop.

Look at attention patterns during decoding. If you see strong attention to the last 2-3 tokens only, that's the problem. Also examine the **causal mask** to ensure it's properly preventing the model from attending to future positions.

For example, if translating "I love cats" produces "J'aime aime aime", the decoder is likely stuck attending to its own output. Solutions include: adjusting temperature, using repetition penalty in decoding, or checking if the model was trained with proper teacher forcing. You might also look at the attention dropout rate, as too much dropout can cause unstable attention patterns.

33.You need to add a new low-resource language into your multilingual model. Would you retrain embeddings, entire encoder, or use adapters? Explain your strategy.

Answer: Use **adapters** as the primary strategy. Adapters are small trainable modules inserted into frozen transformer layers. This preserves existing language capabilities while adding new ones efficiently.

Keep the main model frozen and add language-specific adapter layers (typically 2-5% of model parameters). Train only the adapters and optionally expand the embedding layer for new vocabulary tokens. This prevents catastrophic forgetting of other languages.

For example, adding Hindi to a model knowing 50 languages: freeze the encoder, add new Hindi tokens to embeddings (train only these new rows), insert adapter modules in each layer, and train on Hindi data. The model retains English, French, etc. while learning Hindi. If you have more data, you could do continued pretraining with replay (mixing old and new language data), but adapters are safer and faster.

34.You are building a document search system. Your bi-encoder embeddings don't capture semantic similarity well. How would you debug the embedding space?

Answer: Start by **visualizing embeddings** using t-SNE or UMAP. Similar documents should cluster together. If they don't, you have a problem.

Check specific cases: take known similar documents and compute their cosine similarity. Should be above 0.7-0.8. Look at nearest neighbors, do they make sense semantically?

For example, if searching for "machine learning tutorials", check if results include "AI courses" and "deep learning guides". If nearest neighbors are random, your training data might have weak positive pairs. Debug by: examining training triplets (query, positive, negative), checking if hard negatives are actually different enough, verifying label quality, and testing if the model distinguishes different topics (sports vs technology). You might need better negative sampling or more diverse training data.

35.When would you switch from a Bi-Encoder retriever to a Cross-Encoder re-ranker in a production RAG system?

Answer: Use **Bi-Encoder for initial retrieval** and **Cross-Encoder for re-ranking** when you need both speed and accuracy.

Bi-Encoders are fast because you encode documents once, store embeddings, and do quick similarity search. But they're less accurate because query and document never interact. Cross-Encoders process query and document together, giving better accuracy but too slow for millions of documents.

Production pipeline: Bi-Encoder retrieves top 100 candidates from 10 million documents in milliseconds. Then Cross-Encoder re-ranks these 100 to pick the best 10, taking a few hundred milliseconds. Total time is acceptable.

For example, in a legal document search: Bi-Encoder quickly narrows 5 million cases to 50 relevant ones. Cross-Encoder then carefully compares each with the query to find the most relevant 5 cases, understanding nuanced legal language better.

36.You must process sequences of 80k tokens. Vanilla attention is too slow. Walk through how you would redesign the model using sparse or block attention.

Answer: For 80k tokens, vanilla attention needs 6.4 billion operations per layer. Redesign with **block-sparse attention** combining local and global patterns.

Design strategy: Divide 80k tokens into 160 blocks of 512 tokens each. Each token attends to: its own block (local attention), every 8th block (strided attention), and first/last blocks (global attention). This reduces computation from $O(80k^2)$ to roughly $O(80k \times 2k)$.

Implementation: Layer 1: local window attention (512 tokens). Layer 2: strided attention (every 8th block). Layer 3: local + global. Alternate this pattern through 24 layers.

For example, token at position 40,000 attends to: positions 39,744-40,256 (local), positions 0-512 and 79,488-80,000 (global), and every 4,096th position (strided). This captures both fine-grained local context and important global information while staying fast.

37.Your positional embeddings break when sequence length increases. What changes would you make to fix extrapolation limits?

Answer: Switch from learned absolute positions to **relative position encodings** like RoPE or ALiBi that naturally extrapolate.

RoPE approach: Implement rotary embeddings that encode relative distances between tokens. Train on 4k tokens, it works reasonably well on 8k or 16k.

ALiBi approach: Add a linear bias to attention scores based on distance. Token 10 positions away gets a small negative bias. This requires no position embeddings at all and extrapolates perfectly.

For example, if trained on 2048 tokens but need 8192 at inference: vanilla learned embeddings fail completely (no embedding exists for position 5000). RoPE continues working because it encodes "5 tokens apart" relationships, which it learned during training. Or use ALiBi which penalizes distant tokens consistently regardless of absolute position. You can also try position interpolation (PI), scaling position indices to fit training range.

38.In your model, only 1 out of 12 attention heads seems active. What techniques can you use to encourage head diversity?

Answer: Add **auxiliary losses** that encourage different heads to learn different patterns.

Techniques: Use head diversity loss (penalize similar attention patterns across heads), add orthogonality constraints on head outputs, or apply different dropout rates per head. You can also initialize heads with different random seeds to give them varied starting points.

For example, compute correlation between attention distributions of different heads. If head 1 and head 2 have 0.95 correlation, they're doing the same thing. Add a loss term that penalizes high correlation, forcing heads to diverge.

Another approach: analyze what each head attends to (syntax vs semantics vs position). If all heads show the same pattern, intervene during training. Some practitioners freeze certain heads after initial training to prevent collapse, or use talking heads mechanism where heads share information in a controlled way to maintain diversity.

39.If attention weights collapse to uniform distribution, what could be the reasons and how do you investigate it?

Answer: Uniform attention means the model can't distinguish relevant from irrelevant tokens. Common causes: vanishing attention scores, poor query-key interactions, or optimization issues.

Investigation steps: First, check attention logits before softmax. If all near zero, your queries and keys aren't producing meaningful differences. Look at the norms of Q and K matrices, they might be too small. Check if temperature scaling is too high.

For example, if attention is [0.25, 0.25, 0.25, 0.25] for every token, compute query · key products. If they're all around -0.01 to 0.01, you have insufficient signal. Solutions: increase learning rate for attention parameters, add layer normalization before attention, reduce dropout on attention weights, or initialize Q/K projection matrices with larger variance. Also check if your input embeddings have collapsed (all similar), which would propagate to attention.

40.You trained an MoE model but found that only 2 experts get all the traffic. What techniques help balance expert usage?

Answer: MoE models often suffer from expert imbalance where some experts dominate. Use **load balancing losses** and **capacity factors** to distribute tokens evenly.

Techniques: Add an auxiliary loss that penalizes uneven expert usage (sum of expert frequencies should be uniform). Use a capacity factor (like 1.25) that limits tokens per expert. Implement expert dropout during training to force the model to use backups.

For example, with 8 experts, if expert 1 and 2 handle 90% of tokens: add a load balancing loss that measures how far actual distribution [0.45, 0.45, 0.02, 0.02, 0.02, 0.02, 0.01, 0.01] is from uniform [0.125, 0.125, ...]. Set capacity so each expert can only handle 150 tokens per batch of 1000. Use noise in routing to add randomness. Some models like Switch Transformer use simpler routing (top-1) with strong balancing losses.

41.How do you deploy MoE models efficiently when experts need to be sharded across multiple GPUs?

Answer: Use **expert parallelism** where different experts live on different GPUs, combined with **pipeline parallelism** for other layers.

Strategy: If you have 8 experts and 4 GPUs, put 2 experts per GPU. When processing a batch, tokens are routed to their selected experts across GPUs. This requires all-to-all communication to shuffle tokens between GPUs.

For example, batch has 1000 tokens. Token routing decides: 300 tokens to expert 1 (GPU 0), 250 to expert 3 (GPU 1), etc. Do all-to-all gather to send tokens to correct GPU, each GPU processes its experts locally, then all-to-all scatter to return results. This is communication-heavy but necessary.

Optimizations: use expert replication (same expert on multiple GPUs) for popular experts, compress activations during transfer, overlap communication with computation, or use hierarchical routing (route to GPU first, then expert within GPU).

42.Your seq2seq model's decoder outputs irrelevant content even though encoder embeddings look correct. Where would you trace the fault?

Answer: The problem is likely in **cross-attention** between encoder and decoder. Check if the decoder is actually using encoder information.

Debugging steps: Visualize cross-attention weights. They should focus on relevant encoder positions. If cross-attention is uniform or focuses on wrong tokens, that's your issue. Check if cross-attention is getting gradients during training.

For example, translating "The red car" to French should have cross-attention focusing on "red" when generating "rouge". If it focuses on "The" or spreads uniformly, decoder isn't utilizing encoder properly. Investigate: is cross-attention initialized correctly? Are there too many decoder layers before cross-attention? Is the decoder over-relying on its own self-attention? Try reducing decoder capacity relative to encoder, add more cross-attention layers, or use cross-attention at every decoder layer instead of alternate layers.

43.How do you detect that cross-attention is not effectively utilizing encoder outputs?

Answer: Run **attention analysis** and **ablation tests** to check if cross-attention matters.

Detection methods: Visualize attention maps (should show clear patterns, not uniform). Compute attention entropy (low entropy means focused, high means diffuse). Try zeroing out encoder outputs and see if performance drops significantly. If performance barely changes, cross-attention isn't being used.

For example, in a summarization task, cross-attention should focus on key sentences when generating summary tokens. If you replace encoder outputs with random vectors and the model still works similarly, it's ignoring the encoder. Also check attention gradient magnitudes, if very small, the optimization isn't updating cross-attention effectively. Solutions: increase cross-attention dropout (forces model to rely on it), add auxiliary losses that require encoder information, or pretrain encoder-decoder jointly before fine-tuning.

44.Your generative model outputs good first few tokens but deteriorates after a few steps. How would you diagnose problems in the autoregressive loop?

Answer: This is **exposure bias**, where the model trains on gold tokens but generates its own at inference, causing error accumulation.

Diagnosis: Check perplexity at different generation steps. If it increases quickly, errors compound. Look at attention patterns in later steps, they might become erratic. Examine if the model is attending too much to its own recent (possibly wrong) outputs.

For example, generating "The cat sat on the mat" might start well "The cat sat" but then produce "The cat sat sat sat". Check if: training used too much teacher forcing (always gold context), model wasn't exposed to its own errors during training, or sampling temperature is too high causing drift.

Solutions: Use scheduled sampling (mix gold and generated tokens during training), add reinforcement learning for full sequence training, use nucleus sampling instead of greedy decoding, or implement repetition penalties and length normalization during generation.

45.If the model is hallucinating facts, what adjustments in training objective or decoding strategy might help?

Answer: Hallucination happens when the model generates plausible but incorrect information. Address through **better training objectives** and **constrained decoding**.

Training adjustments: Use factuality-aware training with fact checking signals, add retrieval augmentation (RAG) to ground outputs in real documents, or use contrastive learning to distinguish true vs false statements.

For example, if model claims "Paris is the capital of Germany", in training, provide negative examples with corrections. Use reinforcement learning with factuality rewards, or add a fact verification step in the loss function.

Decoding adjustments: Lower temperature (less creative, more conservative), use nucleus sampling with low p (0.7-0.8), add retrieval during decoding to verify facts, implement confidence thresholds (don't generate if uncertain), or use constrained beam search that only produces facts seen in training or retrieved documents. You can also add explicit uncertainty tokens like "possibly" or "might".

46.You are fine-tuning a multilingual decoder on English-only data. It suddenly loses multilingual ability. How do you prevent language forgetting?

Answer: This is **catastrophic forgetting**. Prevent it by maintaining multilingual data during fine-tuning.

Strategy: Use **data mixing**. Don't fine-tune on English only. Mix 80% English (your target) with 20% other languages (replay data). This keeps multilingual capabilities alive while specializing in English.

For example, fine-tuning for English customer support. Your batches should have: 800 English examples and 200 examples from French, Spanish, German, etc. The model continues seeing diverse languages, preventing forgetting.

Other techniques: Use adapter modules (freeze main model, train adapters only), employ regularization like elastic weight consolidation (EWC) that penalizes changes to important parameters, or use progressive fine-tuning (first adapt on multilingual data with your domain, then specialize). You could also use language-specific adapters or soft prompts instead of full fine-tuning.

47.During fine-tuning on small dataset, the model overfits quickly. What tricks would you use besides dropout?

Answer: Combat overfitting with multiple regularization techniques beyond dropout.

Techniques: Use **early stopping** (stop when validation loss increases), **layer freezing** (freeze lower layers, train only top layers), **data augmentation** (paraphrase, back-translation, synonym replacement), **weight decay** (L2 regularization on parameters), **gradient clipping**, and **learning rate warmup then decay**.

For example, fine-tuning on 500 examples: freeze first 8 layers of a 12-layer model, use weight decay 0.01, add paraphrased versions of training data (doubling dataset size), train with early stopping on validation set of 100 examples. Use smaller learning rate (1e-5 instead of 1e-4). You can also try mixup (blend training examples), label smoothing (soften target distributions), or few-shot prompting instead of full fine-tuning. Using LoRA or adapters also helps by reducing trainable parameters.

48.What is Masked Language Modeling (MLM) and why is it suitable for encoder-only models like BERT? How does masking help the encoder learn bidirectional context?

Answer: MLM randomly masks tokens in input and trains the model to predict them using surrounding context from both directions. About 15% of tokens are masked during training.

It suits encoder-only models because encoders process entire sequences at once, seeing all context. When predicting a masked word, the model uses both left and right context. For example, in "The cat [MASK] on the mat", the model uses "The cat" (left) and "on the mat" (right) to predict "sat".

This creates deep bidirectional understanding. The model learns that "sat" fits between "cat" and "on the mat" by attending to both sides. Without masking, the model could just copy inputs. Masking forces it to understand relationships and context. BERT uses 80% actual [MASK] tokens, 10% random tokens, and 10% unchanged to prevent over-relying on the mask token itself.

49.What problems does standard MLM introduce during inference, and how do models overcome the fact that masks never appear at test time?

Answer: The main problem is **train-test mismatch**. During training, model sees [MASK] tokens but at inference, there are no masks. This gap can hurt performance.

Solutions: BERT uses a mixed masking strategy: 80% time use [MASK] token, 10% replace with random word, 10% keep original word. This reduces dependence on the [MASK] token.

For example, masking "cat" in "The cat sleeps": 80% time becomes "The [MASK] sleeps", 10% time "The dog sleeps", 10% time stays "The cat sleeps". This forces the model to work without explicit mask markers.

At inference, there are no masks at all. The encoder just processes normal text and produces contextualized embeddings. For tasks like classification, you use the [CLS] token embedding. For tasks needing word-level info, you use individual token embeddings. The model has learned rich representations that work without needing masks.

50.What is Causal Language Modeling (CLM) and why is it used in decoder-only models like GPT? Explain how left-to-right prediction shapes model behavior.

Answer: CLM predicts the next token given only previous tokens (left-to-right). Each token can only attend to earlier positions, not future ones. This is enforced by a causal mask in attention.

Decoder-only models like GPT use CLM because they're designed for generation. When generating text, you only have past context, not future. Training matches inference conditions.

For example, given "The cat sat on", predict "the" using only those 4 words. Then predict next word using "The cat sat on the", and so on. The causal mask ensures position 5 never sees position 6 or later.

This shapes behavior to be autoregressive and generative. The model becomes excellent at continuation and generation tasks. It learns to predict what comes next, understanding narrative flow, grammar, and coherence. However, it can't look ahead, limiting its ability to use full context for understanding tasks.

51.Compare MLM and CLM in terms of training objectives, downstream tasks, and the type of architectures they support. Why is MLM poor for generative tasks, while CLM struggles with deep understanding tasks?

Answer:

MLM (Masked Language Modeling):

- Training: Predict masked tokens using bidirectional context
- Architecture: Encoder-only (BERT, RoBERTa)
- Best for: Classification, understanding, question answering
- Context: Full bidirectional access

CLM (Causal Language Modeling):

- Training: Predict next token using left context only
- Architecture: Decoder-only (GPT, LLaMA)
- Best for: Generation, completion, creative writing
- Context: Left-to-right only

Why MLM struggles with generation: MLM is trained to fill in blanks, not generate sequences. It has no practice with autoregressive generation. You'd need to mask iteratively and generate, which is unnatural. BERT can't easily generate stories because it wasn't trained to predict sequentially.

Why CLM struggles with understanding: CLM can't use future context for understanding current position. For classification, it misses right-side information. For question "Is Paris in France?", CLM processes left-to-right and can't look at entire question simultaneously before answering, while MLM sees everything at once, making better judgments about relationships and meaning.